

Handling ensemble lists in Qt-DAB updated

Jan van Katwijk, Lazy Chair Computing
The Netherlands

August 31, 2026

1 Introduction

Qt-DAB - with normal use - shows on its left part a long list of services, services from all ensembles seen. It resembles my car radio, we drove a few times from the Netherlands to Spain and my car radio shows dozens of service names with interesting names, but that obviously cannot be decoded here.

It shows that some form of management of ensemble lists is needed. In Qt-DAB, with its scanning possibilities one may need different service lists for different occasions and Qt-DABe provides a form of management. But first, a note on the modes.

2 Running with different modes

Running the default mode When running with default settings with input from a *real* device, Qt-DAB just collects all names of audio (and EPG/SPI) services and maintains a list of these names between program invocations. (The EPG/SPI services remain invisible and, when the channel such a service is on, the EPG/SPI services runs invisible in the background). On selecting a service, Qt-DAB switches to a different channel - if needed. It may take a few seconds before the data describing the ensemble in that channel is collected.

In the list of services one may set a service to become *favourite*, just by clicking on the third column in the row where the service is found. The favourites - I usually have somewhere between 5 and 10 services marked as favourite - can be shown as a short list, and - obviously - the marking as favourite is stored in the database. (Removing a service from the list of favourites is by clicking on the star in the third column).

It sometimes happens that the content of an ensemble changes, one or more services out, and one or more in. In the current implementation, the *new* services appear in the list, however, the old services remain visible. While in a next version the services list will be updated automatically, in the current version one needs to remove the services from the list manually. Removing a service from the list is by clicking on the first column or the row where the service name is found, i.e. the column where the channelname is displayed.

Running with file input Qt-DAB supports reading from a file. Of course, when running with input from a file, a number of options is meaningless: i.e. selecting a channel, setting a favourite are options with no meaning. In case the channel on which the transmission was recorded was not detected (xml files and ".wav" files generated on Qt-DAB contain the channel information), no channel is shown.

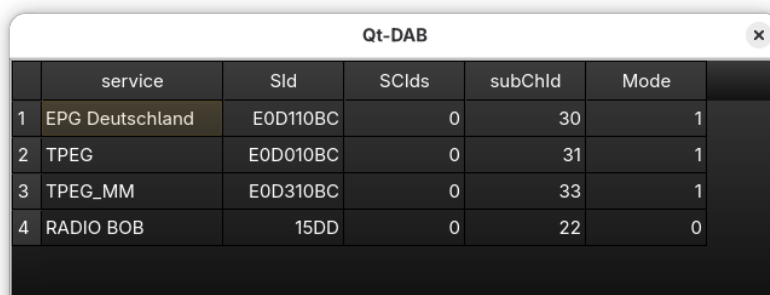
Running with packet services The *configuration and control* window contains a selector, labeled *show only audio services*, a selector by default *set*. If, however, the selector is *unset* there are two things different:

1. Only the services in the ensemble in the currently selected channel are shown, including:
2. packet services. such as TPEG and EPG are shown.

The rationale is that you most likely only want to see packet services is you want to select them. Note that EPG/SPI services are always selected. A selected packet service (usually) does not generate audio, most TPEG type services are encoded in a way Qt-DAB cannot decode them. Qt-DAB spits out the converted packets as udp packets, and from time to time I let a separate program listen to see if further deciphering is possible.

That is why in Qt-DAB the choice was made to run a selected packet service as a background service, just letting it sending output to the aforementioned port. This allows an audio service to be selected as foreground task. (Note that EPG/SPI type services already ran in the background).

The obvious question then is when a started packet service runs in the background, how to stop such a service. Qt-DAB provides a simple controller, showing services, running in background and foreground. Touching the number, given as value for the number of services running (a hidden button on the Configuration and control window), shows a small separate window as shown below



The screenshot shows a window titled "Qt-DAB" with a close button. Inside is a table with 6 columns: an index column, "service", "SId", "SCIds", "subChId", and "Mode". There are 4 rows of data.

	service	SId	SCIds	subChId	Mode
1	EPG Deutschland	E0D110BC	0	30	1
2	TPEG	E0D010BC	0	31	1
3	TPEG_MM	E0D310BC	0	33	1
4	RADIO BOB	15DD	0	22	0

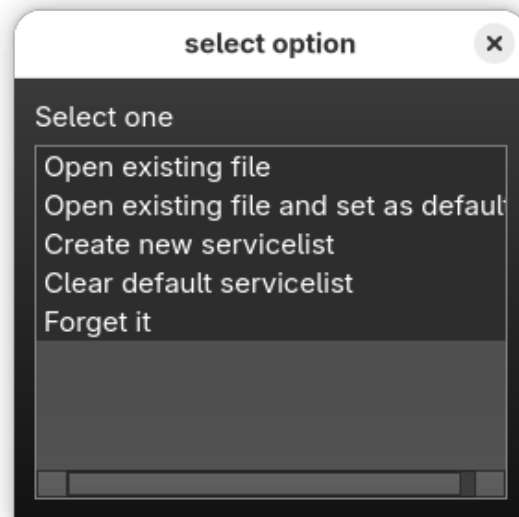
In this example, the table shows that 4 services are running. The foreground service is marked with Mode 0, the background services with Mode 1.

Clicking with the mouse on a background service, either automatically started or by the user, stops that service.

3 The ensemble list

As mentioned, ensemble lists are maintained between program invocations. The default list is stored and an xml encoded file in the directory *serviceLists*, stored in the Qt-DAB-files-7 directory in the home directory.

Setting the selector labeled *load selection* tells Qt-DAB that on the next (re)start you want to choose among a number of options for selecting a serviceList



The options are

1. *Open an existing file.* Apparently in a previous session a named servicelist was created and in the forthcoming session its data is used;
2. *Open an existing file and set as default.* Apparently in a previous session a servicelist was created and from now on it should be used as the default one (The original default one will just remain to exist);
3. *Create new servicelist,* i.e. an empty file will be created that is used during this program invocation as "the" servicelist.
4. *Clear the default servicelist,* with the obvious meaning;
5. *Forget it* just in case you have econd thoughts, in the coming session the default list is used.

Obviously, the color settings are kept between program invocations.

Disclaimer Qt-DAB is developed as hobby program in spare time. Being retired I do have (some) spare time and programming Qt-DAB (and my other programs) is just hobby. It is important to notice that Qt-DAB is - both the sources as the precompiled versions - available *as is*. While it is - obviously - nice if the Qt-DAB software is useful, there are no guarantees, however. If the software does not fit your purpose, or you have problems building an executable, recall that the Qt-DAB software is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

If, however, the software seems useful and you have suggestions for improving and/or extending it, feel free to contact me, suggestions are always welcome (although no guarantees are given that they will be applied.)